



LUND
UNIVERSITY

Numerical Analysis for E3
Numerical Analysis, Centre for Mathematical Sciences

Achim Schroll, April 2003

Homework assignment 4

Delivery date: Tuesday May 6th, 2003, no later than 18:00h.

Topics of this assignment: interpolation with trigonometric function basis; Fast Fourier Transform; filtering and smoothing of functions; Matrix norms.

1. Matrix norm interpretation Consider the unit circle $\mathcal{B} = \{x \in \mathbb{R}^2 : \|x\|_p = 1\}$ and the matrix

$$A = \begin{pmatrix} \sqrt{2} + 1 & 1 \\ 0 & \sqrt{2} \end{pmatrix}.$$

1a) Using MATLAB draw the set $A\mathcal{B} := \{Ax : \forall x \in \mathcal{B}\}$ for $p = 1, 2$ and ∞ .

1b) Explain the meaning of the matrix norm in terms of the sets $\mathcal{B}$ and $A\mathcal{B}$.

2. Inverse Fourier Transform In the course manuscript you find a MATLAB code performing the discrete FFT of a given signal. Write a similar program that performs the inverse FFT. In case you want just modify the code presented in the course, you can download it from the course's exercise page. Download even the file `fft.visa.mat`, which contains the Fourier coefficients of a well-known swedish song. Apply your program to these data and obtain a sound signal. With the MATLAB command `sound(..., 11025)` you can listen to it. Which is the song you hear and who wrote it?

Note, due to round-off errors in the computations you might get a sound signal consisting off complex numbers rather than real ones. Playing such a signal might cause your software to hang, so check the imaginary part of your signal by the MATLAB command `imag`. If it is big, you made a programing error. If it is sufficiently small play the real part only.

3. Orthogonality Prove that $\sum_{j=0}^{N-1} \omega_j^k \omega_j^{-l} = N\delta_{lk}$. This is Theorem 75 in the course manuscript.

4. Filtering and high frequency noise cancellation The transfer function of a filter is often expressed (using the Z -transform) as a rational function of the form

$$H(Z) = \frac{P(Z)}{Q(Z)}$$

where $P(Z)$ and $Q(Z)$ are polynomials on Z . The discrete Fourier transform of the filter is obtained when we substitute $Z = e^{i\omega}$, i.e. we calculate $H(e^{i\omega})$.

Now, to calculate the output of a filter we use the convolution property of the Fourier Transform. This is, for an input signal $x(t)$ and a transfer function $H(e^{i\omega})$, the output signal $y(t)$ is given by the product

$$Y(e^{i\omega}) = H(e^{i\omega})X(e^{i\omega}).$$

4a) Write a Matlab function `filtrering.m` that takes a signal in time $x(t)$, the number of samples of $x(t)$ you want to take in the interval $[0, 1]$, the numerator $P(Z)$ and the denominator $Q(Z)$ and gives you the output in time $y(t)$. Use the build-in *Matlab* functions `fft` and `ifft` to transform $x(t)$ and antitransform $Y(e^{i\omega})$. Test your program using the unitary filter $Q = P \equiv 1$.

4b) Download from the homepage of the course the file `signal.mat` that contains a signal with high frequency noise and the files `cheb.mat` and `butter.mat` that contains the numerator and denominator for a Chebyshev and Butterworth lowpass filters of order 20. Plot the noisy signal and the output of the signal through both filters and compare.

A remark: the implementation of filters of high order requires plenty of components, actives and reactives, with very precise values. In this example, a Butterworth filter of order 20 needs 40+ electronic components with difficult values. In this exercise we replace the circuital implementation with a numerical simulation, possible even on-line because of the speed of the butterfly algorithm.

5. FFT with powers of 3 (Extra exercise!) Write a MATLAB program that calculates the discrete FFT of a signal with a number of samples that is a power of 3. This exercise is not part of the formal requirements of the course.